INTERNET OF THINGS (IoT)

MINI PROJECT REPORT

Arduino UNO Based Four-Wheel Line Following Robot Car

Third Year – Artificial Intelligence & Data Science (AIDS)

Academic Year 2026-27

Submitted By

1. Pranav Ubale
2. Gaurav Suryavanshi
3. Prathamesh Sonawane
4. Pavan Shinde

1. Abstract

This project presents the design and implementation of an autonomous **line following robot car** built around an **Arduino UNO** microcontroller. Two **infrared (IR) sensors** detect a black line on a white surface. Based on the sensor outputs, the Arduino decides whether to move forward, turn left, turn right or stop, and drives four DC gear motors through an **L298N motor driver**. The robot follows the track without any human control and demonstrates the basic IoT building blocks of sensing, processing and actuation.

2. Objectives

- To build a four-wheel robot car that follows a black line automatically.
- To interface IR sensors and a motor driver with an Arduino UNO.
- To control motor direction and speed using digital pins and PWM.
- To understand sensor-based decision making in embedded systems.

3. Components Used

No.	Component	Qty	Purpose in the project
1	Arduino UNO R3 compatible board (SMD ATmega328P, USB-B)	1	Brain of the robot. Reads the sensors and controls the motors.
2	L298N dual H-bridge motor driver	1	Supplies the higher current the motors need and lets the Arduino change their direction and speed.
3	IR sensor module (LM393, 3-pin: VCC, GND, OUT)	2	Detects the black line (left and right of the line).
4	DC gear motors (yellow TT type)	4	Drive the four wheels.
5	Wheels and 4-wheel chassis	1 set	Mechanical body of the car.
6	18650 Li-ion cells (3.7 V each, 2 in series) with holder	2	Power supply (about 7.4 V) for the motors through the L298N.
7	Jumper wires	As needed	Electrical connections between the modules.
8	Black insulation tape on white surface	-	Track (line) for the robot to follow.

4. Working Principle

4.1 IR sensor

Each IR sensor has an IR LED that sends out infrared light and a photodiode that receives the light reflected back. A white surface reflects most of the light, while a black surface absorbs it. A comparator (LM393) on the module converts the received amount into a digital output. A blue potentiometer on the module sets the sensitivity (threshold).

Surface under sensor	Reflected IR	Output LED on module	OUT pin
White	High	ON	LOW (0)
Black tape	Very low	OFF	HIGH (1)

4.2 Decision logic

The two sensors are placed one on each side of the black line. When both see white, the line lies between them and the car goes straight. When one sensor reaches the line, the car turns towards that side to bring the line back to the centre.

Left	Right	Situation	Action	Function in code
LOW	LOW	Both on white, line in the middle	Move forward (speed 100)	moveForward()
LOW	HIGH	Right sensor on black line	Turn right (speed 130)	turnRight()
HIGH	LOW	Left sensor on black line	Turn left (speed 130)	turnLeft()
HIGH	HIGH	Both on black or off track	Stop	stopMotors()

4.3 Motor control through L298N

The L298N has two channels. Each channel has two direction inputs and one enable input. Making one direction input HIGH and the other LOW rotates the motor in one direction, and swapping them reverses it. A PWM signal (`analogWrite`, 0 to 255) on the enable pin controls the speed. Motors on the same side of the car are connected in parallel to one channel, so the car turns by running the two sides in opposite directions (skid steering).

IN1	IN2	Motor A result	IN3	IN4	Motor B result
HIGH	LOW	Forward	HIGH	LOW	Forward
LOW	HIGH	Reverse	LOW	HIGH	Reverse
LOW	LOW	Stop	LOW	LOW	Stop

5. Circuit Diagram

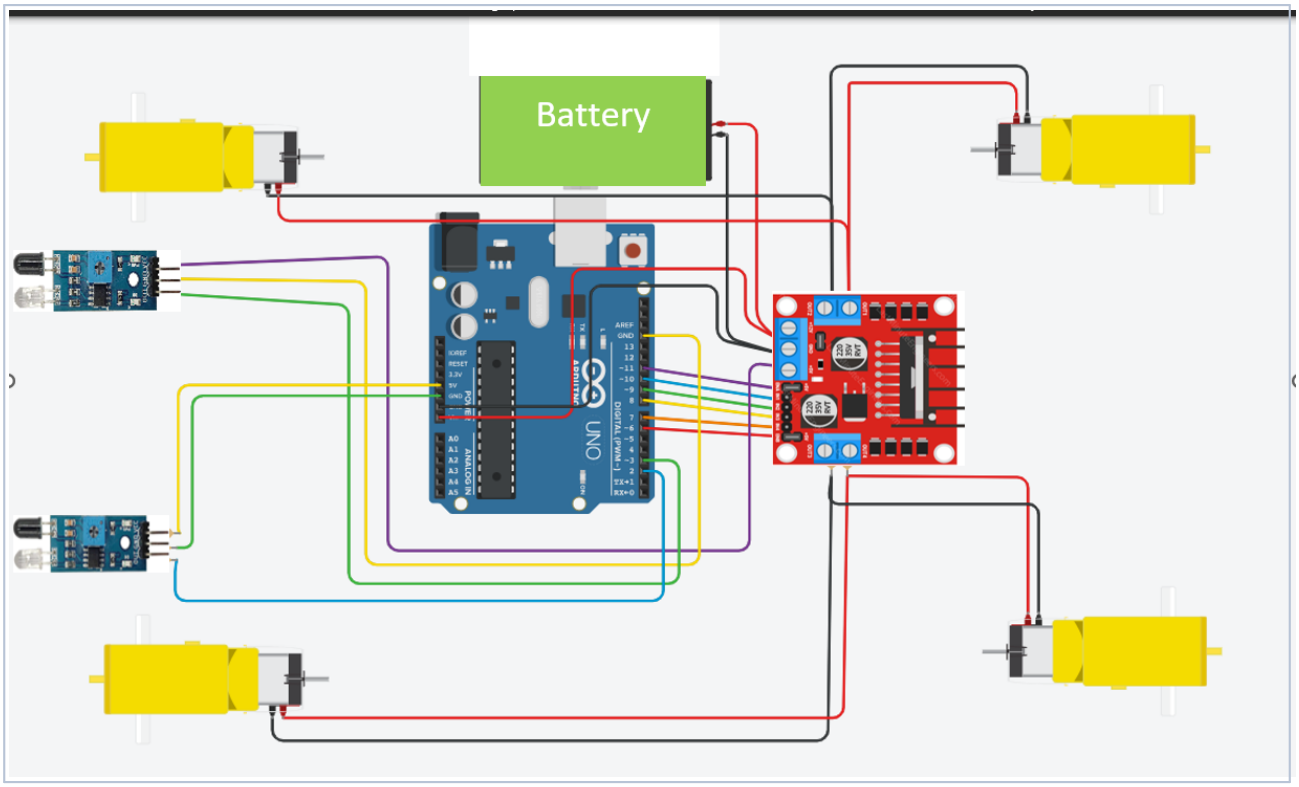


Fig. 1: Circuit diagram of the Arduino UNO based four-wheel line following robot car

The Arduino UNO is in the centre, the two IR sensors are on the left, the L298N motor driver is on the right, and the four motors are placed at the corners of the car. The next section explains every connection one by one.

6. Connections and Reason for Each

6.1 Arduino UNO to L298N motor driver

Arduino pin	L298N pin	Why it is connected
D11 (PWM)	ENA	PWM pin that controls the speed of Motor A (one side of the car).
D10	IN1	Together with IN2, sets the direction of Motor A.
D9	IN2	Together with IN1, sets the direction of Motor A.
D8	IN3	Together with IN4, sets the direction of Motor B.
D7	IN4	Together with IN3, sets the direction of Motor B.
D6 (PWM)	ENB	PWM pin that controls the speed of Motor B (other side of the car).

Note: The jumpers on ENA and ENB must be removed so that the PWM signals from the Arduino control the speed.

6.2 IR sensors to Arduino UNO

Sensor pin	Arduino pin	Why it is connected
Left IR - OUT	D2	Sends the digital line detection signal of the left sensor to the Arduino.
Right IR - OUT	D3	Sends the digital line detection signal of the right sensor to the Arduino.
Both IR - VCC	5V	Powers the sensors from the regulated Arduino 5V supply, which stays stable when the motors run.
Both IR - GND	GND	Completes the sensor power circuit and gives a common reference for the output signal.

6.3 Power connections

From	To	Why it is connected
Battery (+)	L298N 12V terminal	Gives the motors their power (from the 18650 pack) through the driver.
Battery (-)	L298N GND terminal	Completes the motor power circuit.
L298N GND	Arduino GND	Common ground, so the Arduino control signals are read correctly by the driver.
Arduino power input	USB or battery supply	Powers the Arduino board and, through its 5V pin, the IR sensors.

6.4 Motors to L298N

Motors	L298N output	Why it is connected
Two motors on one side (in parallel)	OUT1, OUT2 (Motor A)	Both wheels of that side turn together in the same direction under one channel.
Two motors on the other side (in parallel)	OUT3, OUT4 (Motor B)	Both wheels of that side turn together, and the two sides can be driven independently to steer.

7. Arduino Code

The program reads both sensors in a loop, prints their values on the Serial Monitor for debugging, and calls one of four motor functions depending on the sensor states.

```
// Motor A
const int ENA = 11; // PWM pin for Motor A
const int IN1 = 10;  // IN1 pin for Motor A
const int IN2 = 9;  // IN2 pin for Motor A
#define s 100 //base speed
#define t 130//turning speed


// Motor B
const int ENB = 6; // PWM pin for Motor B
const int IN3 = 8;  // IN3 pin for Motor B
const int IN4 = 7;  // IN4 pin for Motor B

// IR Sensors
const int IRSensorLeft = 2;   // Left IR sensor
const int IRSensorRight = 3;  // Right IR sensor

void setup() {
  // Set motor pins as outputs
  pinMode(ENA, OUTPUT);
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(ENB, OUTPUT);
  pinMode(IN3, OUTPUT);
  pinMode(IN4, OUTPUT);
  // Set sensor pins as inputs
  pinMode(IRSensorLeft, INPUT);
  pinMode(IRSensorRight, INPUT);

  // Initialize serial communication for debugging
  Serial.begin(9600);
}

void loop() {
  bool leftSensor = digitalRead(IRSensorLeft);
  bool rightSensor = digitalRead(IRSensorRight);

  // Print sensor values
  Serial.print("Left Sensor: ");
  Serial.print(leftSensor);
  Serial.print(" | Right Sensor: ");
  Serial.println(rightSensor);

  // Forward movement
  if (leftSensor == LOW && rightSensor == LOW) {
    moveForward();
  }
  // Turn right
  else if (leftSensor == LOW && rightSensor == HIGH) {
```

```
    turnRight();
  }
  // Turn left
  else if (leftSensor == HIGH && rightSensor == LOW) {
    turnLeft();
  }
  // Stop
  else {
    stopMotors();
  }

  delay(100); // Add a short delay to make the print readable
}

void moveForward() {
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  analogWrite(ENA, s); // Adjust speed here

  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);
  analogWrite(ENB, s); // Adjust speed here
}

void turnRight() {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, HIGH);
  analogWrite(ENA, t); // Adjust speed here

  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);
  analogWrite(ENB, t); // Adjust speed here
}

void turnLeft() {
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  analogWrite(ENA, t); // Adjust speed here

  digitalWrite(IN3, LOW);
  digitalWrite(IN4, HIGH);
  analogWrite(ENB, t); // Adjust speed here
}

void stopMotors() {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, LOW);
  analogWrite(ENA, 0); // Stop motor

  digitalWrite(IN3, LOW);
  digitalWrite(IN4, LOW);
  analogWrite(ENB, 0); // Stop motor
}
```

7.1 Code explanation

Part of code	Explanation
Pin constants	Give names to the Arduino pins used for the motor driver and the two IR sensors.
#define s 100, #define t 130	Base speed for moving forward and a higher speed for turning (PWM values out of 255).
setup()	Sets motor pins as outputs, sensor pins as inputs and starts the Serial Monitor at 9600 baud.
loop()	Reads both sensors, prints their values, and chooses forward, right, left or stop.
moveForward()	Both motors forward at speed s.
turnRight() / turnLeft()	Motors on the two sides run in opposite directions at speed t, which rotates the car.
stopMotors()	All direction pins LOW and PWM set to 0.

8. Testing and Problems Faced

The sensor outputs were first checked on the Serial Monitor over white and black surfaces before running the full program.

Problem observed	Cause	Solution
Right IR sensor did not give a reading on the black tape.	The right sensor VCC was connected to the 5V pin of the L298N. Its regulator needs about 7 V or more at the input, and the 2 x 18650 pack (about 7.4 V, lower as it discharges) left it with a low or noisy 5V under motor load.	Both sensors were powered from the Arduino 5V pin. The right sensor then read 1 on black tape correctly.
Sensor output unreliable on black or white.	Sensitivity of the module was not matched to the surface.	The blue potentiometer was adjusted until the output LED turned OFF on black and ON on white.
Unstable sensor readings.	Sensor height or glossy tape.	Sensor kept about 5 to 10 mm above the track, and matte black tape was used.

9. Result

The robot follows the black line on a white surface and corrects its direction automatically using the two IR sensors. It moves forward on a straight line, turns left or right when a sensor meets the line, and stops when both sensors read black.

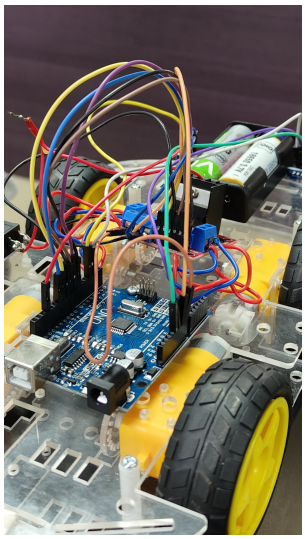


Fig. 2: Assembled prototype showing the Arduino UNO, L298N driver and wiring on the four-wheel chassis

10. Advantages, Limitations and Applications

Advantages	Limitations	Applications
• Low cost • Simple circuit and code • Needs no human control • Easy to extend	• Works only on a high contrast line • Sensitive to ambient light and tape type • Only two sensors, so sharp turns are difficult • Simple on-off control gives jerky motion	• Warehouse and industrial material carriers • Automated guided vehicles (AGV) • Hospital delivery robots • Education and robotics competitions

11. Future Scope

- Add a third or fifth sensor array for smoother and sharper turns.
- Use PID control for stable high speed line following.
- Add an ultrasonic sensor for obstacle avoidance.
- Add Wi-Fi or Bluetooth (ESP8266, ESP32 or HC-05) for remote monitoring and control, making it a full IoT system.

12. Conclusion

The Arduino UNO based four-wheel line following robot car was built and tested successfully. The project covered sensor interfacing, motor driver control with PWM, and decision making in code. A power supply problem with one sensor was identified and solved by powering both sensors from the Arduino 5V pin, which showed the importance of a stable supply and a common ground in embedded circuits.

13. References

- Arduino official documentation: pinMode(), digitalRead(), digitalWrite(), analogWrite() (docs.arduino.cc).
- L298N dual H-bridge motor driver datasheet.
- LM393 comparator based IR sensor module datasheet.